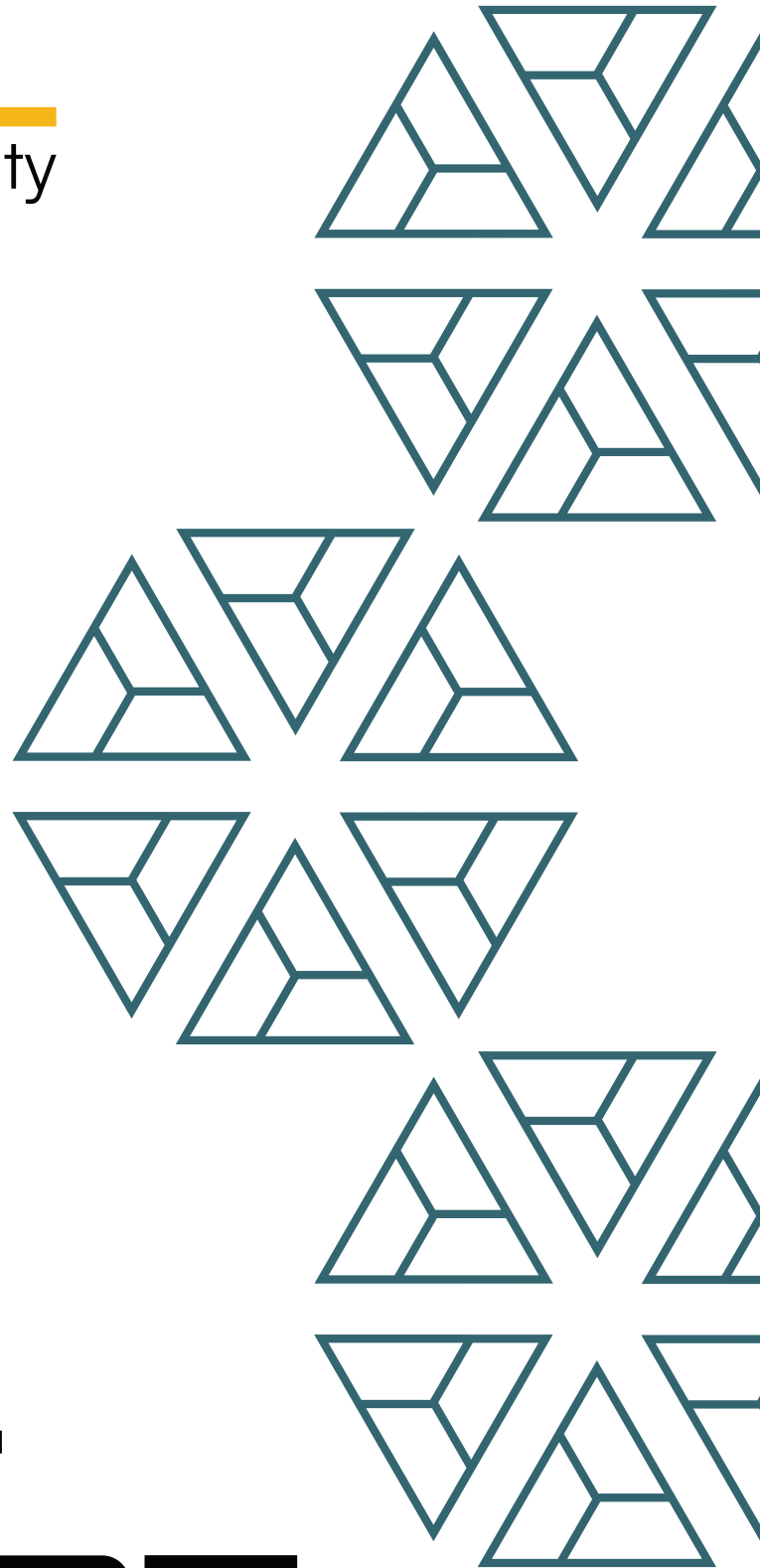




BAIL
security



Parallel Protocol
Upgrade V3.2

FINAL REPORT

April '2026

Disclaimer:

Security assessment projects are time-boxed and often reliant on information that may be provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

The content of this assessment is not an investment. The information provided in this report is for general informational purposes only and is not intended as investment, legal, financial, regulatory, or tax advice. The report is based on a limited review of the materials and documentation provided at the time of the audit, and the audit results may not be complete or identify all possible vulnerabilities or issues. The audit is provided on an "as-is," "where-is," and "as-available" basis, and the use of blockchain technology is subject to unknown risks and flaws.

The audit does not constitute an endorsement of any particular project or team, and we make no warranties, expressed or implied, regarding the accuracy, reliability, completeness, or availability of the report, its content, or any associated services or products. We disclaim all warranties, including the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

We assume no responsibility for any product or service advertised or offered by a third party through the report, any open-source or third-party software, code, libraries, materials, or information linked to, called by, referenced by, or accessible through the report, its content, and the related services and products. We will not be liable for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract.

The contract owner is responsible for making their own decisions based on the audit report and should seek additional professional advice if needed. The audit firm or individual assumes no liability for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract. The contract owner agrees to indemnify and hold harmless the audit firm or individual from any and all claims, damages, expenses, or liabilities arising from the use or reliance on the audit report or the smart contract.

By engaging in a smart contract audit, the contract owner acknowledges and agrees to the terms of this disclaimer.

1. Project Details

Important:

Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Parallel Protocol - Upgrade V3.2 - Audit Report
Website	app.parallel.best
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/parallel-protocol/parallel-tokens/tree/eb3ea38c28051ea5b45ae868930bd88017cc4299 https://github.com/parallel-protocol/parallel-parallelizer/tree/a9d7683d97a4fa522cb83236c5386fb62e752e88
Resolution 1	

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no changes made)	Failed resolution	Open
High						
Medium	1					
Low	2					
Informational	7					
Governance						
Total	10					

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

This audit focuses on an update to the parallel protocol. The update focuses on adding [EIP3009](#) support, allowing meta transactions with non-sequential random nonces. Support is also added for meta transactions for the swap, redeem, and savings functionalities, allowing transactions to be executed by a relayer on behalf of users through signed intents. Lastly, this update patches the ability to put the protocol into a 0-debt state, such that checks triggered during protocol initialization cannot be triggered again. All previous issues flagged before in the previous audit here still apply.

parallel-tokens/EIP3009.sol

[EIP-3009](#) ["Transfer With Authorization"] lets a user sign an off-chain message authorizing a specific token transfer [from -> to, exact value, validity window, random nonce], which any relayer can then submit on-chain via [transferWithAuthorization](#) / [receiveWithAuthorization](#). Unlike [EIP-2612](#), nonces are random and not sequential, allowing parallel execution of multiple transactions without the risk of them reverting due to the order of block inclusion. Furthermore, [EIP-2612](#) only grants allowance, requiring a separate [transferFrom](#) call, while [EIP-3009](#) functions do transfers directly based on signed messages.

Appendix: transferWithAuthorization

This function lets a token holder authorize a one-shot push transfer of a specific amount to a specific recipient via an off-chain signature, which any third party can relay on-chain. The holder signs an [EIP-712](#) message committing to [from, to, value, validAfter, validBefore, nonce]; the contract verifies the signature [ECDSA or [EIP-1271](#)], enforces the validity window, consumes the random nonce in [authorizationStates](#), and executes the transfer. Unlike the other two primitives, anyone may submit this call - there is no restriction on [msg.sender](#), making it suited for gasless relayer payouts where the destination is a passive EOA.

Appendix: receiveWithAuthorization

This function lets a contract atomically pull pre-authorized funds from a user in the same transaction that executes downstream logic [swap, redeem, deposit, etc.]. The signed payload and verification flow are identical to [transferWithAuthorization](#), but with one critical added guard: [msg.sender](#) must equal [to](#). This single constraint eliminates the nonce-burn front-running vector that afflicts the push variant and lets the payee contract bind the pulled value to arbitrary follow-on logic.

Appendix: cancelAuthorization

This function gives the authorizer an escape hatch to invalidate a previously signed authorization before it is redeemed or expires. The authorizer signs a smaller [EIP-712](#) message over just [authorizer, nonce] under a distinct [CancelAuthorization](#) typehash; the contract verifies it and flips the same [authorizationStates\[authorizer\]\[nonce\]](#) slot to true. Unlike the two transfer primitives, it moves no value and has no validity window - it simply pre-consumes the nonce so that any later [transferWithAuthorization](#) or [receiveWithAuthorization](#) carrying the same nonce reverts with [AuthorizationAlreadyUsed](#).

Core Invariants:

INV 1: All authorization functions must ensure the [signer == owner](#).

INV 2: Transactions are only valid if submitted between the [validBefore](#) and [validAfter](#) timestamps.

Issue_01	<code>transferWithAuthorization</code> calls can be denied
Severity	Low
Description	The <code>transferWithAuthorization</code> function can be called by anyone using a valid signature to do the token transfer and consume the nonce. Thus, users monitoring the mempool can see a valid signature and immediately use it by calling <code>transferWithAuthorization</code>, reverting the original transaction since by the time it is included, the nonce is already used up. The issue is that if a contract uses <code>transferWithAuthorization</code> to transfer tokens to itself and then has some further logic to handle said tokens, anyone can frontrun this call with a raw <code>transferWithAuthorization</code> call to just transfer those tokens to the receiver. The post-transfer logic is never run since the original transaction reverts, so the owner is deducted tokens without running logic, which can lead to a loss.
Recommendations	Always use <code>receiveWithAuthorization</code> when using post-transfer logic. However, this is not possible if the transfer is done to some other third account (<code>msg.sender != to</code>). Such cases should not use <code>EIP3009</code>.
Comments / Resolution	

Issue_02	Storage location does not follow EIP7201
Severity	Informational
Description	EIP3009StorageLocation is stored at 0x0f2e86d677e57958274060fd7e3f94ab58d8026e3c78e0e811418ca3fbe98e00. However, evaluating keccak256[abi.encode(uint256(keccak256["cooperlabs.storage.EIP3009"]) - 1)] & ~bytes32(uint256(0xff)) gives 0x34292e02e92581dc72dd1eedcc379a97daab830d016799aa096bf08b96bd9500, which is different. Thus, the storage location does not follow EIP7201.
Recommendations	Change the storage slot or the storage name string used in the hash calculation.
Comments / Resolution	

Issue_03	<code>isValidSignatureNow</code> can be broken for <code>EIP7702</code> accounts
Severity	Informational
Description	The <code>_requireValidSignature</code> function invokes the <code>OpenZeppelin SignatureChecker.isValidSignatureNow</code> function to check signature validity. This function checks the code length to decide whether to check EOA signatures (<code>code.length = 0</code>) or contract signatures. However, if an EOA has enabled <code>EIP7702</code> on their account, that account will have code, so the flow will always go to contract signature checks. If the <code>EIP7702</code> delegatee of that account doesn't implement <code>EIP1271</code>, their signatures can never be validated. This was also raised as an issue in the official OZ repository here, but closed since it is the user's responsibility to ensure proper signature support.
Recommendations	Consider documenting and acknowledging this issue.
Comments / Resolution	

parallel-tokens/TokenP.sol

The `TokenP` contract has been upgraded to support `EIP3009` functions by inheriting the `EIP3009` contract.

No issues were identified in this contract.

parallel-parallelizer/EIP3009.sol

The parallelizer contracts were also updated to support `EIP3009` functionality. This contract is identical to the `EIP3009` contract in the tokens repo already discussed above, and thus inherits the same invariants and security issues flagged previously.

parallel-parallelizer/SavingsEIP3009.sol

The `SavingsEIP3009` contract adds Savings-vault-specific signed-intent authorizations on top of the base `EIP3009` implementation, enabling gasless deposits and redemptions where the signer pre-commits to a specific receiver. The internal `_consumeDepositAuthorization` and `_consumeRedeemAuthorization` helpers verify an `EIP-712` signature over [vault, owner, receiver, assets/shares, validAfter, validBefore, nonce], enforce the validity window, and consume the nonce, letting the inheriting Savings contract pull assets or burn shares on the owner's behalf while guaranteeing the output is routed to the signed receiver. Because both helpers share the inherited `authorizationStates` mapping, each [owner, nonce] tuple is single-use across every authorization type on the contract.

Core Invariants:

INV 1: Nonce used in any one operation[depositauth, redeemauth, transferauth, receiveauth] invalidates that nonce for all other operations.

INV 2: Signatures are only valid between the `validAfter` and `validBefore` timestamps.

No issues were identified in this contract.

parallel-parallelizer/Savings.sol

The upgrade adds two gasless, signature-authorized entry points on top of the inherited `SavingsEIP3009` layer; the rest of the `ERC-4626` surface is unchanged.

`depositWithAuthorization` lets a relayer pull the underlying asset from an owner and mint shares to a signer-specified receiver, all from off-chain signatures, so the depositor pays no gas and cannot have the share recipient redirected.

`redeemWithAuthorization` (exposed in both `[v,r,s]` and `EIP-1271` bytes overloads) lets a relayer burn an owner's shares and deliver the underlying to a signer-specified receiver under the same gasless, front-run-resistant model.

Both are gated by `whenNotPaused` and reuse the existing `_accrue/_convertTo` accounting, so share math and pause semantics are preserved.

Issue_04	Deposit authorizations can be executed at a manipulated share price and mint zero shares
Severity	Informational
Description	The <code>depositWithAuthorization</code> flow lets any caller execute a signed <code>deposit</code> on behalf of the <code>owner</code>, but the signed intent only binds the <code>vault</code>, <code>owner</code>, <code>receiver</code>, <code>asset amount</code>, <code>validity window</code>, and <code>nonce</code>. It does not bind a minimum shares out value or any price bound. When the authorization is finally executed, the contract computes shares from live vault state using floor rounding, so whoever controls the authorization can choose the execution point after the share price has moved against the signer. That creates a timing-sensitive <code>deposit</code> flow where the signer has no slippage protection. If the price per share rises enough before execution, the signer can receive materially fewer shares than expected and, in the worst case, zero shares while the asset transfer still goes through. The dead share seed set during initialization makes the classic empty vault inflation setup harder, but it does not remove the rounding to zero condition, and it does not stop a third party from sitting on an authorization and firing it at the worst available moment.
Recommendations	Consider documenting and acknowledging this behavior.
Comments / Resolution	

parallel-parallelizer/LibAuthorization.sol

The `LibAuthorization` library derives deterministic `EIP-3009` nonces that bind each authorized function in swap or redeem to its full user intent - output to, slippage bounds, deadline, and a `userSalt`. The computed nonce bakes in every intent field (and therefore into the signature), and a relayer cannot alter the destination, slippage, or deadline without invalidating the authorization.

`computeSwapExactInputNonce` / `computeSwapExactOutputNonce` produce the intent-bound nonce for exact-input and exact-output swaps, respectively.

`computeRedeemNonce` does the same for redemptions, additionally committing to the per-collateral `minAmountOuts`.

parallel-parallelizer/Swapper.sol

The `Swapper` contract gains two new authorization-based entry points - `swapExactInputWithAuthorization` and `swapExactOutputWithAuthorization` - that let a relayer execute a mint or burn on behalf of a user via an `EIP-3009` signature, with the user's full swap intent (amounts, to, deadline) baked into the nonce via `LibAuthorization` so it cannot be redirected.

Internally, `_swap` was extended with an `authData` branch that consumes the authorization (pulling tokenIn directly from the signer on mint, or pulling stablecoin into the contract to burnSelf on burn), and a new `_executeAuthorization` helper forwards the call to `IEIP3009.receiveWithAuthorization` and refunds any excess pulled back to the signer.

Additionally, a `CannotBurnAllStableIssued` safety revert was added to the burn path to prevent any swap from draining `normalizedStables` to zero.

Issue_05	<code>receiveWithAuthorization</code> calls are performed without checking token balances
Severity	Medium
Description	In <code>_executeAuthorization</code>, the contract calls <code>receiveWithAuthorization</code> on the token, but doesn't check if any tokens actually get transferred. It hopes that the underlying token contract either reverts (if it doesn't support <code>EIP3009</code>) or transfers the tokens properly if it does, and <code>signatures</code>, <code>balance</code>, etc., are valid. An edge case is encountered in the <code>WETH9</code> contract. If <code>WETH9</code> is ever used as collateral for an eth-stable token, this can cause major issues. This is because the <code>WETH9</code> contract has a fallback function that silently fails if a non-existent function is called. <pre>function() public payable { deposit(); }</pre> So when <code>receiveWithAuthorization</code> is called on <code>WETH</code>, the transaction does not revert, but no <code>WETH</code> is transferred either. If <code>WETH9</code> is called in swap, input tokens won't be charged, but output tokens will get minted to the receiver.
Recommendations	Consider adding a pre and post-balance check to make sure sufficient tokens are transferred in the <code>receiveWithAuthorization</code> call.
Comments / Resolution	

Issue_06	Swapper and redeemer authorization doesn't support all types of contract signatures
Severity	Low
Description	EIP3009 supports EIP1271 contract signatures. EIP1271 contract signatures, however, can be of any length, not necessarily of 65 bytes. This is because IERC1271.isValidSignature calls can also validate multi-sig signatures, taking in multiple signatures and validating all of them together. Thus, if proper EIP1271 support is to be present, the signature must be accepted in a bytes memory and not in v,r,s, which only supports 65-byte signatures. The EIP3009 functions support bytes memory signatures, but the authorization functions in Savings and Redeem, only allow v,r,s style 65-byte signatures. Thus, this does not fully support EIP1271 contract signatures, since they can be of any length, not just 65 bytes.
Recommendations	Consider changing the AuthorizationParams struct to support bytes memory signatures, or document and acknowledge this issue.
Comments / Resolution	

Issue_07	Swap and Redeemed events attribute the relayer instead of the authorizer under the auth flows
Severity	Informational
Description	The Swapper and Redeemer facets emit their main activity events using <code>msg.sender</code> as the <code>from</code> field: <pre>emit Swap[tokenIn, tokenOut, amountIn, amountOut, msg.sender, to];</pre> <pre>emit Redeemed[amount, tokens, amounts, forfeitTokens, msg.sender, to];</pre> In case of authorization-based flows, the from address is <code>params.from</code> and not <code>msg.sender</code>. Thus, in such cases, the events emitted are incorrect and can lead to issues on off-chain scripts monitoring the same.
Recommendations	Propagate the authorizer down to the event emission instead of relying on <code>msg.sender</code> .
Comments / Resolution	

Issue_08	No-op swap signature nonce never consumed
Severity	Informational
Description	In <code>_swap</code>, if <code>amountIn</code> and <code>amountOut</code> are both 0, the entire function is skipped, including <code>_executeAuthorization</code>, which consumes the signature nonce. Thus, if a valid signature is generated for such a transaction with 0 input and output amounts, the swap function can be called, but the nonce will never get consumed. Thus, this nonce can get exposed in a public transaction and then replayed again. Since the input/output amount is 0, the effect of this is highly limited.
Recommendations	Consider documenting and acknowledging this issue, or consume the nonce in the parent authorization function.
Comments / Resolution	

parallel-parallelizer/Redeemer.sol

The `Redeemer` contract gains a new authorization-based entry point - `redeemWithAuthorization` - that lets a relayer execute a redemption on behalf of a user via an EIP-3009 signature on the stablecoin, with the user's full redemption intent (amount, receiver, deadline, per-collateral `minAmountOuts`) baked into the nonce via `LibAuthorization.computeRedeemNonce` so it cannot be redirected.

It pulls the signed amount of stablecoin into the contract via `IEIP3009.receiveWithAuthorization` and then invokes the existing `_redeem` flow with `from = address(this)`, so collateral distribution, whitelist enforcement, and slippage checks remain unchanged. There is also a safety revert in `_updateNormalizer` ensuring a redemption cannot leave `normalizedStables == 0`, preserving the `CannotBurnAllStableIssued` invariant on the redeem path.

Issue_09	<code>redeemWithAuthorization</code> lacks <code>forfeitTokens</code> , breaking gasless redeems on whitelisted collateral
Severity	Informational
Description	<code>redeemWithAuthorization</code> always passes an empty <code>forfeitTokens</code> array into <code>_redeem</code> , so there's no way for a signer to skip a collateral they can't receive. If even one collateral in the redemption flow is whitelist-gated and the receiver isn't on its whitelist, the whole gasless redeem reverts, whereas the non-authorized <code>redeemWithForfeit</code> lets users just forfeit the bad collateral and get the rest. Net effect: on any deployment with a whitelisted collateral in the curve, non-whitelisted users are locked out of the gasless flow and relayers eat gas on guaranteed reverts until that collateral drains.
Recommendations	Consider allowing specifying <code>forfeitTokens</code> in authorization based redemptions as well.
Comments / Resolution	

Issue_10	Renormalization can turn tiny residual issuance into rounding dust and make near-total burns revert
Severity	Informational
Description	When the <code>normalizer</code> becomes too small or too large, the contract renormalizes issuance by recomputing each collateral bucket with integer division. That rescaling floors every per collateral <code>normalizedStables</code> value independently before rebuilding the global total. If the remaining issuance after a burn is already extremely small, those residual amounts can collapse into rounding dust during renormalization. In the decrease path, that can make <code>ts.normalizedStables</code> become zero after rescaling, which then triggers <code>CannotBurnAllStableIssued</code> and reverts the burn. The zeroed value does not persist because the whole transaction reverts, so this is not a state corruption issue. The practical effect is that very small residual issuance may become unburnable at the margin due to accounting precision rather than economic intent.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	